



10-DARS · 14 TADAN

Ma'lumotlar bazasi: SQL va PostgreSQL

Relyatsion bazalar, jadvallar va bog'lanishlar, SQL (SELECT, JOIN, GROUP BY), indekslar, normalizatsiya, tranzaksiyalar, Node.js'dan PostgreSQL bilan ishlash, SQL injection va MongoDB bilan taqqoslash.

O'rta · ~150 daqiqa

Oldingi darsda ma'lumotlarni JSON faylda saqladik. Bu yondashuv ming foydalanuvchida "yiqiladi": qidiruv sekin, bir vaqtdagi yozuvlar to'qnashadi, "shu muallifning 2020-yildan keyingi, narxi 50 mingdan arzon kitoblari" kabi so'rovlarni har safar qo'lda yozish kerak. **Ma'lumotlar bazasi boshqaruv tizimi (DBMS)** bu muammolarni o'nlab yillik muhandislik tajribasi bilan hal qiladi. Bu darsda eng mashhur ochiq manbali relyatsion baza — **PostgreSQL** bilan ishlaymiz.

Relyatsion va NoSQL bazalar

	Relyatsion (SQL)	NoSQL (hujjatli)
Misollar	PostgreSQL, MySQL, SQLite, SQL Server	MongoDB, Firestore, CouchDB
Tuzilma	Jadvallar, qat'iy sxema (ustunlar va turlar)	JSON'ga o'xshash hujjatlar, moslashuvchan sxema
Bog'lanishlar	Kalitlar va JOIN — kuchli	Hujjat ichiga joylash yoki havola — cheklangan
Tranzaksiyalar	To'liq ACID	Cheklangan/qo'shimcha
Qachon tanlash	Aksariyat ilovalar: e-commerce, moliya, CRM, ijtimoiy tarmoqlar	Tuzilmasi tez o'zgaradigan ma'lumot, loglar, kontent katalogi



Tavsiya

Qaysi birini tanlashni bilmasangiz — **PostgreSQL** ni tanlang. U relyatsion ma'lumotlar bilan birga `JSONB` ustunlari orqali hujjatli ma'lumotlarni ham juda yaxshi saqlaydi.

Asosiy tushunchalar

- **Jadval (table)** — bir turdagi obyektlar to'plami: `users`, `books`.
- **Qator (row, yozuv)** — bitta obyekt: bitta kitob.
- **Ustun (column)** — xususiyat va uning turi: `title TEXT`, `price INTEGER`.
- **Birlamchi kalit (PRIMARY KEY)** — har bir qatorni noyob aniqlovchi ustun (odatda `id`).
- **Tashqi kalit (FOREIGN KEY)** — boshqa jadvalning birlamchi kalitiga havola. Bog'lanishlarni yaratadi va yaxlitlikni kafolatlaydi (mavjud bo'lmagan muallifga kitob yozib bo'lmaydi).
- **Sxema (schema)** — jadvallar, ustunlar, turlar va cheklovlarning umumiy tuzilmasi.

Bog'lanish turlari

Tur	Misol	Qanday amalga oshiriladi
Birga-bir (1:1)	Foydalanuvchi ↔ profil	Ikkinchi jadvalda <code>UNIQUE</code> tashqi kalit
Birga-ko'p (1:N)	Muallif → kitoblar	"Ko'p" tomonda tashqi kalit: <code>books.author_id</code>
Ko'pga-ko'p (N:M)	Kitoblar ↔ janrlar, talabalar ↔ kurslar	Oraliq (bog'lovchi) jadval: <code>book_genres(book_id, genre_id)</code>

PostgreSQL'ni o'rnatish

Terminal

```
# Eng oson usul – Docker (14-dars):
docker run --name pg -e POSTGRES_PASSWORD=secret -e POSTGRES_DB=kitoblar -p
5432:5432 -d postgres:17

# Yoki: postgresql.org/download dan o'rnatuvchi; macOS: brew install
postgresql@17

# Konsol mijozi
psql -h localhost -U postgres -d kitoblar
# \l – bazalar    \dt – jadvallar    \d books – jadval tuzilmasi    \q – chiqish
```

Grafik interfeys afzal bo'lsa: **pgAdmin**, **DBeaver**, **TablePlus** yoki VS Code kengaytmalari. Bulutda bepul PostgreSQL: Neon, Supabase.

SQL: jadval yaratish (DDL)

SQL

```
CREATE TABLE authors (  
  id          SERIAL PRIMARY KEY,          -- avtomatik o'suvchi butun  
  son (1, 2, 3...)   
  name        TEXT NOT NULL,   
  country     TEXT DEFAULT '0''zbekiston', -- satr ichidagi '   
  ikkilantiriladi   
  born_year   INTEGER CHECK (born_year > 0),   
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()   
);  
  
CREATE TABLE books (  
  id          SERIAL PRIMARY KEY,   
  title       TEXT NOT NULL,   
  author_id   INTEGER NOT NULL REFERENCES authors(id) ON DELETE RESTRICT,   
  price       INTEGER NOT NULL CHECK (price ≥ 0), -- so'mda, butun son   
  stock       INTEGER NOT NULL DEFAULT 0 CHECK (stock ≥ 0),   
  isbn        TEXT UNIQUE,   
  published   DATE,   
  meta        JSONB DEFAULT '{}'::jsonb,          -- moslashuvchan   
  qo'shimcha ma'lumot   
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()   
);  
  
ALTER TABLE books ADD COLUMN pages INTEGER;  
ALTER TABLE books RENAME COLUMN pages TO page_count;  
DROP TABLE IF EXISTS temp_data;
```

Asosiy ma'lumot turlari

Tur	Ishlatilishi
INTEGER, BIGINT	Butun sonlar; SERIAL / BIGSERIAL yoki GENERATED ALWAYS AS IDENTITY — avto-ID
NUMERIC(12,2)	Aniq o'nlik sonlar — pul uchun (FLOAT emas!)

<code>TEXT</code> , <code>VARCHAR(n)</code>	Matn
<code>BOOLEAN</code>	true/false
<code>DATE</code> , <code>TIMESTAMPTZ</code>	Sana; vaqt zonasi bilan sana-vaqt (doim <code>TIMESTAMPTZ</code> ni tanlang)
<code>UUID</code>	Global noyob ID: <code>gen_random_uuid()</code>
<code>JSONB</code>	JSON hujjat, indekslash va so'rov bilan
<code>TEXT[]</code>	Massiv

Cheklovlar (constraints)

`NOT NULL`, `UNIQUE`, `CHECK`, `PRIMARY KEY`, `FOREIGN KEY` — bu baza darajasidagi **oxirgi himoya chizig'i**. Ilova kodida xato bo'lsa ham, baza manfiy narx yoki mavjud bo'lmagan muallifni saqlashga yo'l qo'ymaydi.

SQL: ma'lumot bilan ishlash (CRUD)

SQL

```
-- CREATE: qo'shish
INSERT INTO authors (name, born_year) VALUES
  ('Abdulla Qodiriy', 1894),
  ('Cho'lpon', 1897),
  ('Robert Martin', 1952);

INSERT INTO books (title, author_id, price, stock, published)
VALUES ('O'tkan kunlar', 1, 65000, 12, '1926-01-01')
RETURNING id, title;                                -- yaratilgan qatorni darhol
qaytaradi

-- READ: o'qish
SELECT * FROM books;
SELECT title, price FROM books WHERE price < 100000 ORDER BY price DESC LIMIT
10;
SELECT * FROM books WHERE title ILIKE '%kun%';        -- katta-kichik harfsiz
qidiruv
SELECT * FROM books WHERE author_id IN (1, 2) AND stock > 0;
SELECT * FROM books WHERE published BETWEEN '1920-01-01' AND '1940-12-31';
SELECT * FROM books WHERE isbn IS NULL;
SELECT DISTINCT country FROM authors;

-- Sahifalash: 3-sahifa, har sahifada 20 ta
SELECT * FROM books ORDER BY id LIMIT 20 OFFSET 40;

-- UPDATE: yangilash (WHERE ni UNUTMANG – aks holda BARCHA qatorlar o'zgaradi!)
UPDATE books SET price = 59000, stock = stock - 1 WHERE id = 1 RETURNING *;

-- DELETE: o'chirish (WHERE ni UNUTMANG!)
DELETE FROM books WHERE stock = 0 AND published < '1900-01-01';
```

JOIN — jadvallarni birlashtirish

Relyatsion bazalarning asosiy kuchi. Kitob va muallif nomini birga olish:

SQL

```
-- INNER JOIN: faqat ikkala jadvalda mos keladigan qatorlar
SELECT b.title, a.name AS author, b.price
FROM books b
JOIN authors a ON a.id = b.author_id
ORDER BY a.name;

-- LEFT JOIN: chap jadvalning HAMMA qatorlari (o'ng tomonda mos bo'lmasa -
NULL)
-- "Barcha mualliflar va kitoblari, kitobi yo'qlari ham"
SELECT a.name, b.title
FROM authors a
LEFT JOIN books b ON b.author_id = a.id;

-- Kitobi yo'q mualliflar
SELECT a.name
FROM authors a
LEFT JOIN books b ON b.author_id = a.id
WHERE b.id IS NULL;
```

JOIN turi	Natija
INNER JOIN	Faqat ikkala tomonda mos kelganlar (kesishma)
LEFT JOIN	Chapdagi hammasi + o'ngdan mos kelganlar
RIGHT JOIN	O'ngdagi hammasi + chapdan mos kelganlar (kam ishlatiladi)
FULL JOIN	Ikkala tomonning hammasi

Agregatsiya: GROUP BY va HAVING

SQL

```
-- Umumiy statistika
SELECT COUNT(*) AS jami, AVG(price)::int AS ortacha, MIN(price), MAX(price),
SUM(price * stock) AS ombor_qiymati
FROM books;

-- Har bir muallif bo'yicha
SELECT a.name, COUNT(b.id) AS kitoblar, COALESCE(SUM(b.stock), 0) AS nusxalar
FROM authors a
LEFT JOIN books b ON b.author_id = a.id
GROUP BY a.id, a.name
HAVING COUNT(b.id) ≥ 2           -- HAVING – guruhlardan keyin filtr (WHERE
-- guruhlashdan oldin)
ORDER BY kitoblar DESC;

-- Bajarilish tartibi: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER
BY → LIMIT
```

Subquery va CTE

SQL

```
-- O'rtacha narxdan qimmat kitoblar
SELECT title, price FROM books
WHERE price > (SELECT AVG(price) FROM books);

-- CTE (WITH) – murakkab so'rovni nomlangan qadamlarga bo'lish
WITH oylik AS (
    SELECT date_trunc('month', created_at) AS oy, SUM(total) AS daromad
    FROM orders
    WHERE status = 'paid'
    GROUP BY 1
)
SELECT oy, daromad,
       daromad - LAG(daromad) OVER (ORDER BY oy) AS oldingi_oyga_nisbatan
FROM oylik
ORDER BY oy;
```

Normalizatsiya

Normalizatsiya — ma'lumotlarni takrorlanmaydigan va ziddiyatsiz qilib jadvallarga bo'lish. Yomon dizayn:

order_id	customer_name	customer_phone	products
1	Ali Valiyev	+998901112233	Kitob A, Kitob B
2	Ali Valiyev	+998901112233	Kitob C

Muammolar: Ali telefonini o'zgartirsa, uni barcha buyurtmalarda yangilash kerak (birida unutilsa — ziddiyat); "Kitob A nechta sotildi?" degan savolga matnni parse qilmasdan javob berib bo'lmaydi. To'g'ri dizayn — har bir mavjudot alohida jadvalda, bog'lanishlar kalitlar orqali:

Matn

```
customers(id, name, phone)
orders(id, customer_id → customers, status, total, created_at)
order_items(order_id → orders, book_id → books, qty, price)  -- narx sotilgan
paytdagi qiymat!
```

Asosiy qoidalar (1NF–3NF): har bir katakda bitta qiymat (ro'yxat emas); har bir ustun to'liq birlamchi kalitga bog'liq; kalit bo'lmagan ustunlar bir-biriga bog'liq emas. Amaliy qoida: *"bitta fakt — bitta joyda"*.

 `order_items.price` ataylab takrorlangan: kitob narxi keyin o'zgarsa ham, eski buyurtmada sotilgan narx saqlanib qolishi kerak. Bu — ongli denormalizatsiya.

Indekslar

Indekssiz baza `WHERE email = '...'` so'rovi uchun **har bir qatorni** tekshiradi (Seq Scan). Indeks — kitob oxiridagi alifbo ko'rsatkichi kabi: kerakli joyni darhol topadi (B-tree, $O(\log n)$).

SQL

```
CREATE INDEX idx_books_author ON books(author_id);           -- JOIN va filtrlar uchun
CREATE UNIQUE INDEX idx_users_email ON users(lower(email)); -- katta-kichik harfsiz noyoblik
CREATE INDEX idx_orders_user_date ON orders(user_id, created_at DESC); -- kompozit

-- So'rov rejasini ko'rish
EXPLAIN ANALYZE SELECT * FROM books WHERE author_id = 5;
-- Index Scan using idx_books_author ... (actual time=0.02..0.03 ms)
```

- **PRIMARY KEY** va **UNIQUE** avtomatik indekslanadi; **tashqi kalitlar** — **yo'q**, ularni qo'lda indekslang.
- Indeks o'qishni tezlashtiradi, lekin yozishni biroz sekinlashtiradi va joy egallaydi — faqat haqiqatan so'raladigan ustunlarga qo'ying.

Tranzaksiyalar va ACID

Tranzaksiya — bir nechta amalni "**hammasi yoki hech biri**" tamoyili bilan bajarish. Klassik misol — pul o'tkazmasi: birinchi hisobdan yechildi, ikkinchisiga yozishda server o'chib qoldi — pul "yo'qoladi". Tranzaksiya buni oldini oladi.

SQL

```
BEGIN;
  UPDATE accounts SET balance = balance - 500000 WHERE id = 1;
  UPDATE accounts SET balance = balance + 500000 WHERE id = 2;
COMMIT;           -- ikkalasi birga saqlanadi
-- yoki xato bo'lsa: ROLLBACK; - hech narsa o'zgarmaydi
```

- **Atomicity** — bo'linmaslik: hammasi yoki hech biri.
- **Consistency** — izchillik: cheklovlar (**CHECK balance ≥ 0**) har doim bajariladi.
- **Isolation** — ajratilganlik: parallel tranzaksiyalar bir-birining yarim natijasini ko'rmaydi.
- **Durability** — barqarorlik: **COMMIT** dan keyin ma'lumot, hatto elektr o'chsa ham, saqlanadi.

Node.js'dan PostgreSQL bilan ishlash

Terminal

```
npm install pg
```

src/db/pool.js

```
import pg from 'pg';

// Pool — tayyor ulanishlar to'plami (har so'rovga yangi ulanish ochish qimmat)
export const pool = new pg.Pool({
  connectionString: process.env.DATABASE_URL, //
  postgres://user:pass@host:5432/db
  max: 10,
});

export const query = (text, params) => pool.query(text, params);
```

JavaScript

```
import { query } from './db/pool.js';

const { rows } = await query('SELECT id, title, price FROM books WHERE price < $1 ORDER BY price', [100000]);
console.log(rows); // [{ id: 1, title: "O'tkan kunlar", price: 65000 }, ...]

const { rows: [book] } = await query(
  'INSERT INTO books (title, author_id, price) VALUES ($1, $2, $3) RETURNING *',
  ['Kecha va kunduz', 2, 55000]
);
```

SQL injection — eng xavfli xato

JavaScript

```
// ❌ HECH QACHON: foydalanuvchi ma'lumotini SQL satriga qo'shmang
const email = req.body.email; // hujumchi yuboradi: ' OR '1'='1
await query(`SELECT * FROM users WHERE email = '${email}'`);
// Natijaviy SQL: SELECT * FROM users WHERE email = ' OR '1'='1' → BARCHA
// foydalanuvchilar!
// Yoki: '; DROP TABLE users; -- → jadval o'chiriladi

// ✅ DOIM parametrlangan so'rovlar ($1, $2 ...)
await query('SELECT * FROM users WHERE email = $1', [email]);
// Baza qiymatni hech qachon SQL kodi sifatida talqin qilmaydi
```



Qat'iy qoida

SQL'dagi har bir o'zgaruvchan qiymat — faqat `$1, $2...` parametrlari orqali. Ustun yoki saralash nomini dinamik tanlash kerak bo'lsa — oldindan belgilangan ruxsat ro'yxatidan (whitelist) foydalaning: `const col = { price: 'price', title: 'title' }[req.query.sort] ?? 'id'.`

ORM va query builder'lar

Xom SQL o'rniga ko'pincha **ORM** (Object-Relational Mapping) ishlatiladi: **Prisma**, **Drizzle**, **Sequelize**, **TypeORM**. Ular sxemani kodda tasvirlash, migratsiyalar va qulay API beradi. Lekin SQL'ni bilish baribir zarur — ORM qanday so'rov hosil qilayotganini tushunish va sekin so'rovlarni tuzatish uchun.

JavaScript

```
// Prisma misoli — xuddi shu so'rov
const books = await prisma.book.findMany({
  where: { price: { lt: 100000 }, author: { country: "O'zbekiston" } },
  include: { author: true },
  orderBy: { price: 'asc' },
});
```

Amaliy misollar

MISOL 1

Blog platformasi sxemasi va real so'rovlar

Foydalanuvchilar, postlar, izohlar, teglar (N:M) va layklar bo'lgan blog uchun to'liq sxema.

```

CREATE TABLE users (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  username    TEXT NOT NULL UNIQUE CHECK (username ~ '^[a-z0-9_]{3,30}$'),
  email       TEXT NOT NULL,
  password_hash TEXT NOT NULL,
  bio         TEXT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX users_email_uq ON users (lower(email));

CREATE TABLE posts (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  author_id   BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  title       TEXT NOT NULL CHECK (length(title) BETWEEN 3 AND 200),
  slug       TEXT NOT NULL UNIQUE,
  body       TEXT NOT NULL,
  status      TEXT NOT NULL DEFAULT 'draft' CHECK (status IN ('draft',
'published')),
  published_at TIMESTAMPTZ,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX posts_author_idx ON posts (author_id);
CREATE INDEX posts_published_idx ON posts (published_at DESC) WHERE status
= 'published'; -- qisman indeks

CREATE TABLE comments (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  post_id     BIGINT NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
  author_id   BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  body       TEXT NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX comments_post_idx ON comments (post_id, created_at);

CREATE TABLE tags (
  id         INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name      TEXT NOT NULL UNIQUE
);

CREATE TABLE post_tags (                                -- ko'pga-ko'p bog'lovchi
jadvall
  post_id BIGINT REFERENCES posts(id) ON DELETE CASCADE,

```

```
tag_id INT REFERENCES tags(id) ON DELETE CASCADE,  
PRIMARY KEY (post_id, tag_id)  
);  
  
CREATE TABLE likes (  
  user_id BIGINT REFERENCES users(id) ON DELETE CASCADE,  
  post_id BIGINT REFERENCES posts(id) ON DELETE CASCADE,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  PRIMARY KEY (user_id, post_id)          -- bir foydalanuvchi bir  
postga bir marta  
);
```

```

-- 1. Bosh sahifa lentasi: muallif, izohlar va layklar soni, teglar
SELECT p.id, p.title, p.slug, p.published_at,
       u.username AS author,
       (SELECT COUNT(*) FROM comments c WHERE c.post_id = p.id) AS
comments,
       (SELECT COUNT(*) FROM likes l WHERE l.post_id = p.id) AS likes,
       COALESCE(array_agg(t.name ORDER BY t.name) FILTER (WHERE t.id IS
NOT NULL), '{}') AS tags
FROM posts p
JOIN users u ON u.id = p.author_id
LEFT JOIN post_tags pt ON pt.post_id = p.id
LEFT JOIN tags t ON t.id = pt.tag_id
WHERE p.status = 'published'
GROUP BY p.id, u.username
ORDER BY p.published_at DESC
LIMIT 10;

-- 2. "javascript" tegidagi postlar
SELECT p.title FROM posts p
JOIN post_tags pt ON pt.post_id = p.id
JOIN tags t ON t.id = pt.tag_id
WHERE t.name = 'javascript' AND p.status = 'published';

-- 3. Oxirgi 30 kundagi eng faol 5 muallif (postlar + izohlar)
WITH activity AS (
  SELECT author_id, 'post' AS kind FROM posts WHERE created_at > now() -
interval '30 days'
  UNION ALL
  SELECT author_id, 'comment' FROM comments WHERE created_at > now() -
interval '30 days'
)
SELECT u.username,
       COUNT(*) FILTER (WHERE kind = 'post') AS posts,
       COUNT(*) FILTER (WHERE kind = 'comment') AS comments,
       COUNT(*) AS total
FROM activity a JOIN users u ON u.id = a.author_id
GROUP BY u.username
ORDER BY total DESC
LIMIT 5;

-- 4. Layk qo'yish (takroriy bo'lsa - xato emas, shunchaki e'tiborsiz)
INSERT INTO likes (user_id, post_id) VALUES (7, 42) ON CONFLICT DO

```

```
NOTHING;
```

```
-- 5. Post qidiruvi (to'liq matnli qidiruv)
```

```
SELECT title, ts_rank(to_tsvector('simple', title || ' ' || body), q) AS  
rank  
FROM posts, plainto_tsquery('simple', 'react hook') q  
WHERE to_tsvector('simple', title || ' ' || body) @@ q  
ORDER BY rank DESC;
```

`ON DELETE CASCADE` — foydalanuvchi o'chirilsa, uning postlari, izohlari va layklari ham avtomatik o'chadi. `FILTER (WHERE ...)` — bitta so'rovda bir nechta shartli hisob. Kompozit birlamchi kalit `(user_id, post_id)` takroriy laykni baza darajasida taqiqlaydi.

MISOL 2 Express API'ni JSON fayldan PostgreSQL'ga ko'chirish

9-darsdagi qatlamli arxitektura o'z samarasini beradi: **faqat service qatlami** o'zgaradi, routes va controllers tegilmaydi.

migrations/001_books.sql

```
CREATE TABLE IF NOT EXISTS books (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  title       TEXT NOT NULL,  
  author      TEXT NOT NULL,  
  price       INTEGER NOT NULL CHECK (price > 0),  
  stock       INTEGER NOT NULL DEFAULT 0 CHECK (stock ≥ 0),  
  genre       TEXT NOT NULL,  
  year        INTEGER,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  
  updated_at  TIMESTAMPTZ,  
  UNIQUE (title, author)  
);  
CREATE INDEX IF NOT EXISTS books_genre_idx ON books (genre);
```

```

import { query } from '../db/pool.js';
import { HttpError } from '../utils/errors.js';

const SORT_SQL = {                                // whitelist – dinamik ORDER BY
  uchun xavfsiz usul
  price: 'price ASC',
  '-price': 'price DESC',
  title: 'title ASC',
  '-createdAt': 'created_at DESC',
};

const COLUMNS = `id, title, author, price, stock, genre, year,
  created_at AS "createdAt", updated_at AS "updatedAt"`;

export async function listBooks({ q, genre, minPrice, maxPrice, sort,
  page, limit }) {
  const where = [];
  const params = [];
  const add = (sql, value) => { params.push(value);
  where.push(sql.replaceAll('?', `$$${params.length}`)); };

  if (q) add('(title ILIKE ? OR author ILIKE ?)', `%${q}%`); // ikkala ?
  → bitta $n
  if (genre) add('genre = ?', genre);
  if (minPrice !== undefined) add('price ≥ ?', minPrice);
  if (maxPrice !== undefined) add('price ≤ ?', maxPrice);

  const whereSql = where.length ? `WHERE ${where.join(' AND ')}` : '';
  const offset = (page - 1) * limit;

  const [{ rows }, { rows: [{ total }] }] = await Promise.all([
    query(
      `SELECT ${COLUMNS} FROM books ${whereSql}
      ORDER BY ${SORT_SQL[sort]} LIMIT $$${params.length + 1} OFFSET
      $$${params.length + 2}`,
      [...params, limit, offset]
    ),
    query(`SELECT COUNT(*)::int AS total FROM books ${whereSql}`, params),
  ]);

  return { data: rows, meta: { page, limit, total, pages: Math.ceil(total
  / limit) } };

```

```

}

export async function getBook(id) {
  const { rows: [book] } = await query(`SELECT ${COLUMNS} FROM books WHERE
id = $1`, [id]);
  if (!book) throw HttpError.notFound('Kitob topilmadi');
  return book;
}

export async function createBook({ title, author, price, stock, genre,
year }) {
  try {
    const { rows: [book] } = await query(
      `INSERT INTO books (title, author, price, stock, genre, year)
      VALUES ($1, $2, $3, $4, $5, $6) RETURNING ${COLUMNS}`,
      [title, author, price, stock, genre, year ?? null]
    );
    return book;
  } catch (err) {
    if (err.code === '23505') throw HttpError.conflict('Bu kitob
allaqachon mavjud'); // unique_violation
    throw err;
  }
}

export async function updateBook(id, patch) {
  const allowed = ['title', 'author', 'price', 'stock', 'genre', 'year'];
  const entries = Object.entries(patch).filter(([k]) =>
allowed.includes(k));
  if (!entries.length) return getBook(id);

  const set = entries.map(([k], i) => `${k} = ${i + 2}`).join(', ');
  const { rows: [book] } = await query(
    `UPDATE books SET ${set}, updated_at = now() WHERE id = $1 RETURNING
${COLUMNS}`,
    [id, ...entries.map([, v]) => v])
  );
  if (!book) throw HttpError.notFound('Kitob topilmadi');
  return book;
}

export async function deleteBook(id) {
  const { rowCount } = await query('DELETE FROM books WHERE id = $1',
[id]);

```

```
if (!rowCount) throw HttpError.notFound('Kitob topilmadi');  
}
```

`add()` yordamchisi qiymatni `params` ga qo'shib, shartdagi `?` belgilarini tegishli `$n` ga almashtiradi (`q` shartida ikkala `?` bitta parametrni ishlatadi). Ustun nomlari faqat `allowed` ro'yxatidan, qiymatlar esa faqat parametrlar orqali — injection'ga joy yo'q. UUID formatida bo'lmagan `id` kelsa, PostgreSQL `22P02` xatosini beradi — uni ham `errorHandler` da 404/400 ga aylantirish mumkin.

MISOL 3 Buyurtma berish tranzaksiyasi (ombor, pool client, qulflash)

Buyurtma: (1) har bir kitob omborda yetarlimi, (2) ombordan ayirish, (3) buyurtma va uning qatorlarini yozish. Agar biror qadam muvaffaqiyatsiz bo'lsa — **hech narsa** o'zgarmasligi kerak.

SQL

```
CREATE TABLE orders (  
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  user_id     BIGINT NOT NULL,  
  total       INTEGER NOT NULL,  
  status      TEXT NOT NULL DEFAULT 'pending',  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE TABLE order_items (  
  order_id BIGINT REFERENCES orders(id) ON DELETE CASCADE,  
  book_id  UUID REFERENCES books(id),  
  qty      INTEGER NOT NULL CHECK (qty > 0),  
  price    INTEGER NOT NULL,  
  PRIMARY KEY (order_id, book_id)  
);
```

```

import { pool } from '../db/pool.js';
import { HttpError } from '../utils/errors.js';

// Qayta ishlatiladigan tranzaksiya yordamchisi
export async function withTransaction(fn) {
  const client = await pool.connect(); // tranzaksiya BITTA ulanishda
  bo'lishi shart
  try {
    await client.query('BEGIN');
    const result = await fn(client);
    await client.query('COMMIT');
    return result;
  } catch (err) {
    await client.query('ROLLBACK');
    throw err;
  } finally {
    client.release(); // ulanishni pool'ga qaytarish
  }
}

export function placeOrder(userId, items) { // items: [{ bookId, qty }]
  return withTransaction(async (db) => {
    const ids = items.map(i => i.bookId);

    // FOR UPDATE – bu qatorlarni tranzaksiya tugaguncha qulflaydi:
    // ikki xaridor oxirgi nusxani bir vaqtda sotib o'lolmaydi
    const { rows: books } = await db.query(
      'SELECT id, title, price, stock FROM books WHERE id = ANY($1) ORDER
      BY id FOR UPDATE',
      [ids]
    );
    const byId = new Map(books.map(b => [b.id, b]));

    let total = 0;
    for (const { bookId, qty } of items) {
      const book = byId.get(bookId);
      if (!book) throw HttpError.notFound(`Kitob topilmadi: ${bookId}`);
      if (book.stock < qty) {
        throw HttpError.conflict(`${book.title}" omborda yetarli emas
        (bor: ${book.stock}, so'raldi: ${qty})`);
      }
    }
  }
}

```

```

    total += book.price * qty;
  }

  const { rows: [order] } = await db.query(
    'INSERT INTO orders (user_id, total) VALUES ($1, $2) RETURNING *',
    [userId, total]
  );

  for (const { bookId, qty } of items) {
    await db.query('UPDATE books SET stock = stock - $1 WHERE id = $2',
      [qty, bookId]);
    await db.query(
      'INSERT INTO order_items (order_id, book_id, qty, price) VALUES
      ($1, $2, $3, $4)',
      [order.id, bookId, qty, byId.get(bookId).price]
    );
  }

  return { ...order, items };
});
}

```

Uchta muhim nuqta: tranzaksiya uchun `pool.query` emas, bitta `client` kerak (aks holda har bir so'rov boshqa ulanishga tushishi mumkin); `FOR UPDATE` "race condition" ni yo'q qiladi; `ORDER BY id` bilan qulflash tartibi bir xil bo'lgani uchun "deadlock" ehtimoli kamayadi. Xato tashlansa — `ROLLBACK`, ombor qiymatlari o'zgarmaydi.

MISOL 4 Taqqoslash uchun: xuddi shu model MongoDB + Mongoose'da

NoSQL yondashuvni his qilish uchun blog postini MongoDB'da modellashtiramiz. Bu yerda izohlar post hujjati **ichiga joylanadi**.

Terminal

```

npm install mongoose
# docker run -d --name mongo -p 27017:27017 mongo:8

```

```

import mongoose from 'mongoose';

const commentSchema = new mongoose.Schema({
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
true },
  body: { type: String, required: true, maxlength: 2000 },
}, { timestamps: true });

const postSchema = new mongoose.Schema({
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
true, index: true },
  title: { type: String, required: true, trim: true, minlength: 3,
maxlength: 200 },
  slug: { type: String, required: true, unique: true },
  body: String,
  tags: [{ type: String, lowercase: true }], // massiv - alohida
jadval shart emas
  status: { type: String, enum: ['draft', 'published'], default: 'draft'
},
  comments: [commentSchema], // ichma-ich
hujjatlar
  likes: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],
}, { timestamps: true });

postSchema.index({ tags: 1, createdAt: -1 });
postSchema.virtual('likeCount').get(function () { return
this.likes.length; });

export const Post = mongoose.model('Post', postSchema);

```

JavaScript

```
import mongoose from 'mongoose';
import { Post } from './models/Post.js';

await mongoose.connect('mongodb://localhost:27017/blog');

// Yaratish
const post = await Post.create({
  author: userId, title: 'React hook\'lari', slug: 'react-hooks', tags:
['react', 'JavaScript'],
});

// "javascript" tegidagi nashr qilingan postlar, muallif ma'lumoti bilan
(JOIN o'xshashi)
const posts = await Post.find({ tags: 'javascript', status: 'published' })
  .populate('author', 'username')
  .sort({ createdAt: -1 })
  .limit(10)
  .lean();

// Izoh qo'shish – atomar operator
await Post.updateOne({ _id: post._id }, { $push: { comments: { author:
userId, body: 'Zo\'r!' } } });

// Layk (takrorlanmas) – $addToSet
await Post.updateOne({ _id: post._id }, { $addToSet: { likes: userId } });

// Agregatsiya: teglar bo'yicha postlar soni
const stats = await Post.aggregate([
  { $match: { status: 'published' } },
  { $unwind: '$tags' },
  { $group: { _id: '$tags', count: { $sum: 1 } } },
  { $sort: { count: -1 } },
]);
```

Jihat	PostgreSQL	MongoDB
Izohlar	Alohida jadval + JOIN	Post ichida (bitta o'qishda keladi, lekin hujjat 16 MB bilan cheklangan)
Teglar	N:M jadval	Satrlar massivi

Sxema	Baza darajasida majburiy	Mongoose (ilova darajasida)
"Foydalanuvchining barcha izohlari"	Oddiy <code>WHERE</code> <code>author_id</code>	Barcha postlarni skanerlash yoki agregatsiya — noqulay

Xulosa: ma'lumotlarni qanday **so'rashingiz** modelni belgilaydi. Bog'lanishlar ko'p bo'lsa — relyatsion baza tabiiyroq.

Mashqlar

Mustaqil ish

1. SQLBolt interaktiv darslarini to'liq bajaring.
2. Onlayn do'kon sxemasini loyihalang: `categories` (ichma-ich, `parent_id`), `products`, `customers`, `orders`, `order_items`, `reviews` (1–5 baho).
3. Shu sxema uchun so'rovlar: eng ko'p sotilgan 10 mahsulot; har bir toifaning o'rtacha reytingi; hech narsa sotib olmagan mijozlar; oylik daromad.
4. `EXPLAIN ANALYZE` bilan indeksli va indeksiz so'rov tezligini 100 000 qatorli jadvalda solishtiring (`generate_series` bilan test ma'lumot yarating).
5. 3-misoldagi `placeOrder` ni sinang: ikki terminaldan bir vaqtda oxirgi nusxani sotib olishga harakat qiling. `FOR UPDATE` olib tashlansa nima bo'ladi?

▼ 4-mashq uchun test ma'lumot

SQL

```
INSERT INTO books (title, author, price, stock, genre)
SELECT 'Kitob ' || g, 'Muallif ' || (g % 500), (random() * 200000 +
10000)::int, (random() * 50)::int,
      (ARRAY['roman','ilmiy','bolalar','dasturlash','tarix'])[1 + g %
5]
FROM generate_series(1, 100000) g;
```

✓ Xulosa

- Relyatsion baza: jadvallar, birlamchi va tashqi kalitlar, cheklovlar — ma'lumot yaxlitligining kafolati.
- SQL: `SELECT ... JOIN ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT .`
- Normalizatsiya: "bitta fakt — bitta joyda"; N:M uchun bog'lovchi jadval.
- Tez-tez filtrlanadigan va JOIN qilinadigan ustunlarga indeks; `EXPLAIN ANALYZE` bilan tekshiring.
- Bir nechta bog'liq yozuv — tranzaksiya ichida (`BEGIN/COMMIT/ROLLBACK`).
- Faqat parametrlangan so'rovlar — SQL injection'dan himoya.

☐ Darsni o'qib bo'ldim